МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Моделирование сред и разработка приложений виртуальной и дополненной реальности»

Практическая работа № 2

Студент группы *ИКБО-50-23, Враженко Д.О.* _____
(подпись)

Старший преподаватель *Горбатов Г.В.* _____
(подпись)

Отчет представлен «___»_____202__г.

Москва 2026 г.

1. Использование систем: Transform; Rigidbody и Collider

В практической работе были использованы различные системы, такие как **Transform**, **Rigidbody** и **Collider**. На рисунке 1 представлен пример их интеграции в итоговый проект на объекте Player.

Если рассматривать эти компоненты не как инструменты кода, а как категории бытия цифрового объекта, то их триединство описывает классическую структуру сущего. **Transform** отвечает на экзистенциальный вопрос о положении в пространстве. **Collider** очерчивает границы его естества, отделяя внутреннее содержание от внешнего хаоса среды; это его форма и идентичность, ответ на вопрос "Что я такое?". **Rigidbody** — это диалог души и мира, принцип существования в пространстве. Это ответ на вопрос "Как я взаимодействую с реальностью?", подчиняюсь ли я ее законам или сопротивляюсь им, воплощенный в массе, силе и инерции.

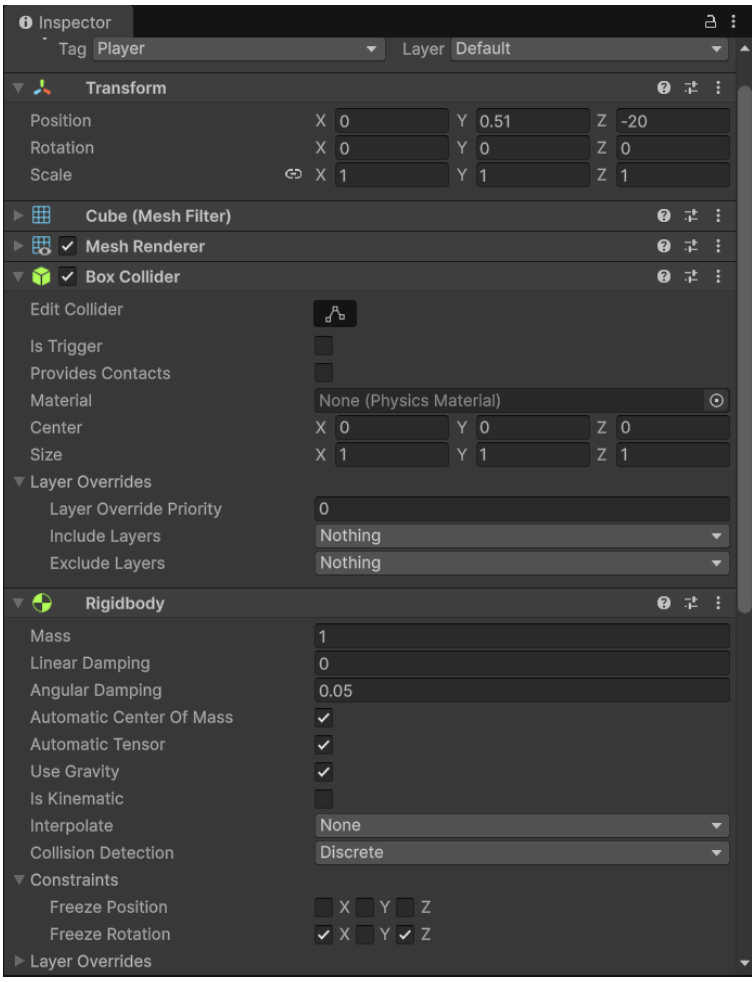

Рисунок 1 — Использование физических систем

2. Создание пользовательского контроллера

Был реализован пользовательский контроллер, то есть специальный компонент, который описывает управление движением персонажа и камерой. В листинге 1 приведен код с контроллером персонажа, а в листинге 2 — камеры.

Листинг 1 - Код скрипта PlayerController.cs

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    [Header("Движение")]
    public float moveSpeed = 8f;
    public float acceleration = 10f;

    [Header("Прыжок")]
    public float jumpForce = 8f;
    public float jumpCutMultiplier = 0.5f;

    [Header("Механики платформера")]
    public float coyoteTime = 0.15f;
    public float jumpBufferTime = 0.15f;

    Rigidbody rb;
    float coyoteTimer;
    float jumpBufferTimer;
    bool isGrounded;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        HandleJumpInput();
        HandleJumpLogic();
    }

    void FixedUpdate()
    {
        Move();
    }

    void Move()
    {
        float x = Input.GetAxis("Horizontal");
        float z = Input.GetAxis("Vertical");
        Vector3 move = transform.right * x + transform.forward * z;
        rb.linearVelocity = new Vector3(move.x * moveSpeed,
rb.linearVelocity.y, move.z * moveSpeed);
    }

    void HandleJumpInput()
    {
        if (Input.GetButtonDown("Jump"))
            jumpBufferTimer = jumpBufferTime;
        else
```

```
            jumpBufferTimer -= Time.deltaTime;
        if (Input.GetButtonUp("Jump") && rb.linearVelocity.y > 0)
            rb.linearVelocity = new Vector3(rb.linearVelocity.x,
rb.linearVelocity.y * jumpCutMultiplier, rb.linearVelocity.z);
    }

    void HandleJumpLogic()
    {
        if (isGrounded)
            coyoteTimer = coyoteTime;
        else
            coyoteTimer -= Time.deltaTime;
        if (jumpBufferTimer > 0 && coyoteTimer > 0)
        {
            rb.linearVelocity = new Vector3(rb.linearVelocity.x, jumpForce,
rb.linearVelocity.z);
            jumpBufferTimer = 0;
        }
    }

    void OnCollisionStay(Collision collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
            isGrounded = true;
    }

    void OnCollisionExit(Collision collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
            isGrounded = false;
    }
}
```

Листинг 2 - Код скрипта CameraController.cs

```
using UnityEngine;

public class CameraController : MonoBehaviour
{
    [Header("Характеристики")]
    public Transform player;
    public float mouseSensitivity = 200f;
    float xRotation = 0f;

    void Start()
    {
        Cursor.lockState = CursorLockMode.Locked;
    }

    void Update()
    {
        float mouseX = Input.GetAxis("Mouse X") * mouseSensitivity *
Time.deltaTime;
        float mouseY = Input.GetAxis("Mouse Y") * mouseSensitivity *
Time.deltaTime;
        xRotation -= mouseY;
        xRotation = Mathf.Clamp(xRotation, -70f, 70f);
        transform.localRotation = Quaternion.Euler(xRotation, 0f, 0f);
        player.Rotate(Vector3.up * mouseX);
    }
}
```

3. Работа с переменными персонажа; Создание навигационной системы и объекта противника или NPC

Работа с переменными персонажа представлена на рисунке 2.

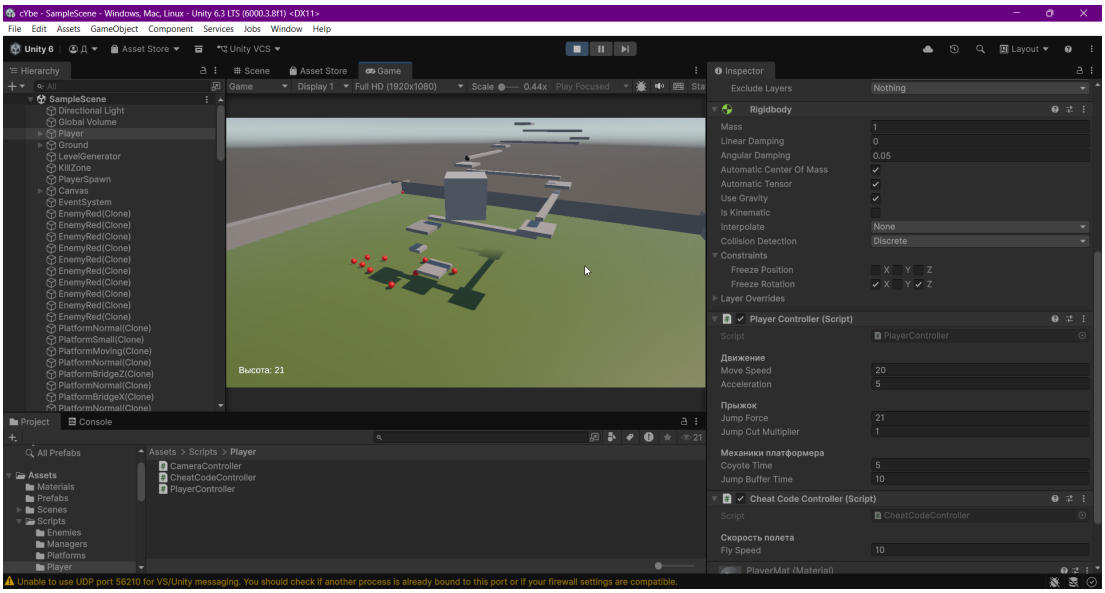

Рисунок 2 — Работа с параметрами персонажа

Навигационная система для поверхности продемонстрирована на рисунке 3.

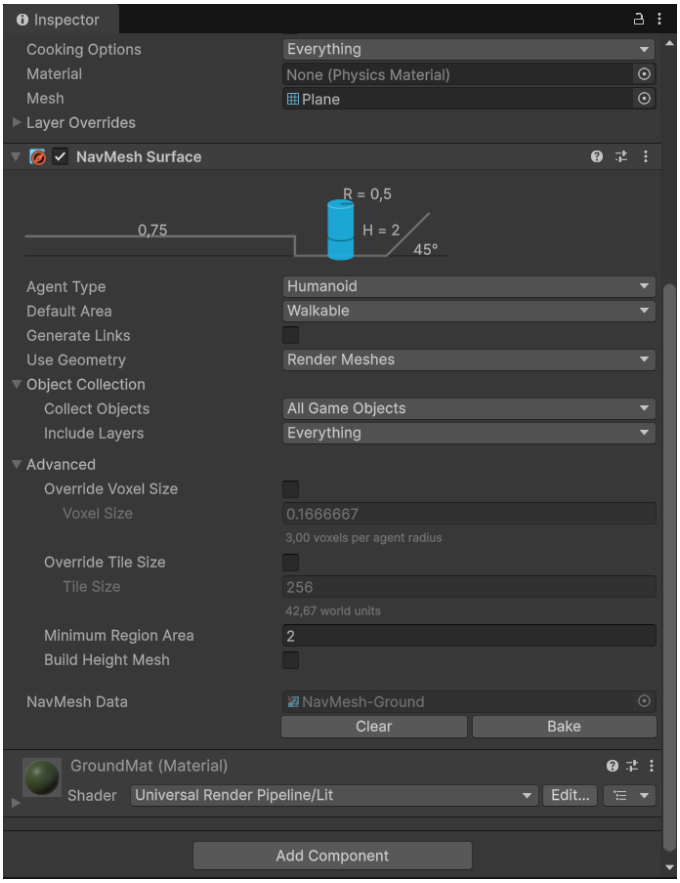

Рисунок 3 — Навигационная система для поверхности

Навигационная система для противника продемонстрирована на рисунке 4.

Рисунок 4 — Навигационная система для противника